Knowledge Representation and Reasoning
Project 4: Actions with Cost

Jakub Seliga, Michał Szewczak, Sebastian Trojan, Wiktoria Koniecko

Introduction

The field of Knowledge Representation and Reasoning (KRR) focuses on formalizing information so that a computer system can utilize it to solve complex tasks, such as simulating the evolution of a dynamic environment. Within this domain, action languages provide a structured way to model how autonomous agents interact with their surroundings through sequences of operations. By defining the logic behind state transitions, these systems enable automated reasoning about the consequences, feasibility, and resource requirements of specific plans. The primary objective of this project is to formalize and implement a logic-based framework for representing and reasoning about dynamic systems within the $\mathbb{DS}_4$ class. The class is an extension of class of dynamic systems described by Action Language $\mathbb{A}$, and has following assumptions:

1. **Inertia law:** Fluents persist in their state unless changed by an action.
2. **Complete information:** The system assumes full knowledge of all actions and all fluents.
3. **Branching model of time:** The temporal structure allows for multiple possible future paths.
4. **Deterministic and sequential actions:** Only one action occurs at a time, and every action has a single, predictable outcome.
5. **Characteristics of actions:**
   - *Precondition:* A set of literals representing the conditions for execution; if not met, the action results in an empty effect.
   - *Postcondition:* The specific effect of an action, represented as a literal.
   - *Cost:* A cost $\kappa \geq 1$ is required to perform an action. However, $\kappa = 0$ if the cost is unspecified or if the action results in an empty effect.
6. **Universal performance:** In every state, all actions are performed.
7. **Partial descriptions:** The system allows for incomplete descriptions of any given state.
8. **No constraints:** No state or domain constraints are admitted in this class.

The difference from standard Action Language $\mathbb{A}$ is the cost associated with an action.

1 Action language syntax

Here $\mathcal{A}c$ is the set of all actions and $\mathcal{F}$ is the set of all fluents.

1.1 Value statement

$$\bar{f} \text{ after } A_1, \ldots, A_n$$

Where $\bar{f} \in \mathcal{F}$ is a fluent and $A_1, \ldots, A_n \in \mathcal{A}c$ are actions.
Abbreviation when $n = 0$: initially $\bar{f}$

1.2 Effect statement

$$A \text{ causes } \bar{f} \text{ if } g_1, \ldots, g_k$$

Where $\bar{f}, g_1, \ldots, g_k \in \mathcal{F}$ are fluents and $A \in \mathcal{A}c$ is an action.
Abbreviation when $k = 0$: A_1 causes $\bar{f}$

1.3 Cost statement

$$A \text{ costs } \kappa$$

Where $A \in \mathcal{A}c$ is an action and $\kappa > 0$.

2 Action language semantics

A **signature** Υ consists of:

- A finite set of fluents $\mathcal{F}$
- A finite set of actions $\mathcal{A}c$

A non-empty set D of value/effect/cost statements is called an **action domain**. Let $D_{trans} \subseteq D$ be the set of all value and effect statements in D, and $D_{cost} \subseteq D$ be the set of all cost statements in D.

Model construction is based solely on D_{trans}. Cost statements D_{cost} play no role in determining whether a structure is a model; they are used only in a second phase, after the transition function Ψ has been fully determined, to label transitions with their costs.

A **transition function** is a mapping $\Psi : \mathcal{A}c \times \Sigma \to \Sigma$. The value $\Psi(A, \sigma)$ is the state resulting from performing action A in state σ. In case of the $\mathbb{DS}4$ class of dynamic systems the mapping Ψ must be defined for all actions and states because performing an action is always possible.

Let $\mathcal{L}$ be an action language for a $\mathbb{DS}4$ dynamic system. A **structure for** $\mathcal{L}$ is a pair $S = (\Psi, \sigma_0)$, where:

- Ψ is a transition function
- $\sigma_0 \in \Sigma$ is the initial state

A statement $s \in D_{trans}$ **is true in structure** S (in symbols $S \models s$) iff:

- s is of the form

  $$\bar{f} \text{ after } A_1, \ldots, A_n$$

  and $\Psi^\star((A_1, \ldots, A_n), \sigma_0) \models \bar{f}$

- s is of the form

  $$A \text{ causes } \bar{f} \text{ if } g_1, \ldots, g_k$$

  and for every $\sigma \in \Sigma$ such that $\sigma \models g_i$ for all i: $\Psi(A, \sigma) \models \bar{f}$

A structure $S = (\Psi, \sigma_0)$ is a **model** of action domain D in a language over the signature $\Upsilon = (\mathcal{F}, \mathcal{A}c)$ iff:

- $(\forall s \in D_{trans})\ S \models s$
- for every effect statement s of the form

  $$A \text{ causes } \bar{f} \text{ if } g_1, \ldots, g_k$$

  and for every $\sigma \in \Sigma$, if one of the following holds:

  - $s \in D_{trans}$ and $(\exists i \in \{1, \ldots, k\})\ \sigma \not\models g_i$
  - $s \notin D_{trans}$

  then $\sigma \models \bar{f}$ iff $\Psi(A, \sigma) \models \bar{f}$

Let $S = (\Psi, \sigma_0)$ be a model of D (determined using D_{trans} only). The **cost function** $Cost : \mathcal{A}c \times \Sigma \to \mathbb{N}$ is then uniquely determined by Ψ and D_{cost} as follows:

$$Cost(A, \sigma) = \begin{cases} \kappa & \text{if } (A \text{ costs } \kappa) \in D_{cost} \text{ and } \Psi(A, \sigma) \neq \sigma \\ 0 & \text{otherwise} \end{cases}$$

The *otherwise* branch covers two cases mandated by assumption A5: (a) no cost statement for A appears in D_{cost}, and (b) A has an empty effect in state σ (i.e. $\Psi(A, \sigma) = \sigma$). In both cases $Cost(A, \sigma) = 0$.

Since $Cost$ is uniquely determined by Ψ and D_{cost}, we write the full structure as a triple $S = (\Psi, \sigma_0, Cost)$ as a notational convenience, understanding $Cost$ as a derived component.

A **program** P is a sequence of actions to be performed starting from the initial state. The cost of a program $P = (A_1, \ldots, A_k)$ is defined as:

$$Cost^\star((A_1, \ldots, A_k)) = \sum_{i=1}^{k} Cost(A_i,\ \Psi^\star((A_1, \ldots, A_{i-1}), \sigma_0))$$

where $\Psi^\star((), \sigma_0) = \sigma_0$ by convention.

3 Query Language statements

Q1: Goal satisfaction

$$\gamma \text{ after } P$$

Where γ is the goal condition (a set of literals) and P is a program.

Q2: Executability with Cost

$$P \text{ executable with cost } \kappa$$

Where $\kappa > 0$ and P is a program.

Q3: Executability with exact Cost

$$P \text{ executable with exact cost } \kappa$$

Where $\kappa > 0$ and P is a program.

4 Semantics of the query language

4.1 Partial Initial States

A partial initial state represents a set of complete states (completions), each assigning a truth value to every fluent while remaining consistent with the given literals.
Execution of a program P from a partial state σ_0 induces a set of possible models:

$$S_i = (\Psi^i, \sigma_0^i, Cost)$$

Q1: Goal satisfaction

A query of the form:

$$\gamma \text{ after } P$$

is a consequence of action domain D iff for every model $S_i = (\Psi^i, \sigma_0^i, Cost)$:

$$(\forall \bar{f} \in \gamma) \quad \Psi^i(P, \sigma_0^i) \models \bar{f}$$

Q2: Executability with Cost

A query of the form:

$$P \text{ executable with cost } \kappa$$

is a consequence of action domain D iff for every model $S_i = (\Psi^i, \sigma_0^i, Cost)$:

$$Cost(P, \sigma_0^i) \leq \kappa$$

Q3: Executability with exact Cost

A query of the form:

$$P \text{ executable with exact cost } \kappa$$

is a consequence of action domain D iff for every model $S_i = (\Psi^i, \sigma_0^i, Cost)$:

$$Cost(P, \sigma_0^i) = \kappa$$

5 Examples

Przykłady do rozszezrenia i obliczenia do dodania!

Example 1

Let's consider the following statements:

$$\text{initially } \neg doorOpen$$

$$openDoor \text{ causes } doorOpen \text{ if } hasKey$$

$$openDoor \text{ costs } 5$$

Program

$$P = (openDoor)$$

Completions of σ_0

$$\sigma_0^1 = \{\neg doorOpen, hasKey\}$$
$$\sigma_0^2 = \{\neg doorOpen, \neg hasKey\}$$

Execution

Case 1: σ_0^1 Precondition holds, so:

$$\sigma_1^1 = \{doorOpen, hasKey\}$$

Cost:

$$Cost^\star(P) = 5$$

Case 2: σ_0^2 Precondition does not hold, so:

$$\sigma_1^2 = \{\neg doorOpen, \neg hasKey\}$$

Cost:

$$Cost^\star(P) = 0$$

Final States

$$\{\sigma_1^1, \sigma_1^2\}$$

Query Results

Q1: Goal Satisfaction

$$\{doorOpen\} \text{ after } P$$

is false because *doorOpen* does not hold in all final states.

Q2 and Q3: Executability with Cost

$$P \text{ executable with exact cost } 5$$

is false since $Cost^{\star}(P)$ is not always 5.

$$P \text{ executable with cost } 5$$

is true since for all executions:

$$Cost^{\star}(P) \leq 5$$

$$P \text{ executable with cost } 0$$

is false since there exists an execution with cost 5.

Example 2

Let's consider the following statements:

$$\text{initially } \neg doorOpen$$

$$\text{initially } hasKey$$

$$openDoor \text{ causes } doorOpen \text{ if } hasKey$$

$$openDoor \text{ costs } 5$$

Program

$$P = (openDoor, openDoor)$$

Execution

Initial state:

$$\sigma_0 = \{\neg doorOpen, hasKey\}$$

Action 1 Precondition holds, so:

$$\sigma_1 = \{doorOpen, hasKey\}$$

Cost:

$$Cost(openDoor, \sigma_0) = 5$$

Action 2 Precondition holds, but no change occurs:

$$\sigma_2 = \{doorOpen, hasKey\}$$

Cost:

$$Cost(openDoor, \sigma_1) = 0$$

Total Cost

$$Cost^{\star}(P) = 5$$

Query Results

Q2: Executability with Cost

$$P \text{ executable with cost } 5$$

is true.

Q3: Executability with exact Cost

$$P \text{ executable with exact cost } 10$$

is false since:

$$Cost^{\star}(P) = 5 \neq 10.$$

Example 3

Let's consider the following statements:

$$\text{initially } \neg doorOpen$$

$$doorOpen \text{ after } openDoor$$

$$openDoor \text{ causes } doorOpen \text{ if } hasKey$$

$$openDoor \text{ costs } 5$$

Program

$$P = (openDoor)$$

Completions of σ_0

$$\sigma_0^1 = \{\neg doorOpen, hasKey\}$$
$$\sigma_0^2 = \{\neg doorOpen, \neg hasKey\}$$

However, σ_0^2 is not consistent with the action domain.
Indeed, since $hasKey$ does not hold, the effect does not apply, and by inertia:

$$\Psi(openDoor, \sigma_0^2) \models \neg doorOpen$$

But the value statement requires:

$$\Psi(openDoor, \sigma_0^2) \models doorOpen$$

This is a contradiction. Therefore, σ_0^2 is not allowed.

Execution

Thus, the only possible initial state is:

$$\sigma_0 = \{\neg doorOpen, hasKey\}$$

Precondition holds, so:

$$\sigma_1 = \{doorOpen, hasKey\}$$

Cost:

$$Cost^{\star}(P) = 5$$

Final States

$$\{\sigma_1\}$$

Query Results

Q1: Goal Satisfaction

$$\{doorOpen\} \text{ after } P$$

is true.

Q2: Executability with Cost

$$P \text{ executable with cost } 5$$

is true.

Q3: Executability with exact Cost

$$P \text{ executable with exact cost } 5$$

is true.

Example 4

Let's consider the following statements:

$$\text{initially } \neg doorOpen$$

$$\text{initially } \neg hasKey$$

$$doorOpen \text{ after } openDoor$$

$$openDoor \text{ causes } doorOpen \text{ if } hasKey$$

$$openDoor \text{ costs } 5$$

Program

$$P = (openDoor)$$

Execution

There is no model satisfying all statements, because:

- $\sigma_0 \models \neg hasKey$
- the effect requires $hasKey$
- the value statement enforces $doorOpen$ after $openDoor$

These constraints are inconsistent.

Query Results

All queries are undefined since no model exists.

6 Technical description

The compiler was implemented in Python as a pipeline with five main stages: input normalization, parsing, model construction, query evaluation, and result presentation. The implementation follows the formal semantics of the $\mathbb{DS}_4$ action language used in this project.

6.1 Data structures

The source specification is first normalized by removing comments and optional trailing dots. Every non-empty line is stored together with its original line number, which allows the compiler to report precise parsing errors.

The internal representation is built from several small logical structures:

- **Literal** stores a fluent and its polarity, for example *doorOpen* or !*doorOpen*.
- **Value statement** stores a literal and a program, representing either an initial fact or a statement of the form f after $A_1, \ldots, A_n$.
- **Effect statement** stores an action, a resulting literal, and a tuple of preconditions.
- **Cost statement** stores an action and its declared cost.
- **Query structures** represent the three supported query types: goal queries, bounded-cost queries, and exact-cost queries.

At the domain level, statements are grouped into three tuples: value statements, effect statements, and cost statements. This makes later validation and evaluation simpler.

6.2 Representation of states and programs

The set of fluents and actions is inferred automatically from the domain. After that, each fluent is assigned an index. A complete state is represented as a tuple of Boolean values. If the fluent order is $(doorOpen, hasKey, alarmActive)$, then the state $(False, True, False)$ corresponds to $\{\neg doorOpen, hasKey, \neg alarmActive\}$.

To access values efficiently, the compiler stores a dictionary mapping each fluent name to its position in the state tuple. Programs are represented as tuples of action names.

6.3 Transition and cost tables

The core semantic structures are:

- a **transition table** indexed by $(action, state)$, returning the next state,
- a **cost table** indexed by $(action, state)$, returning the cost of the action in that state,
- a tuple of valid initial states, interpreted as the set of models of the action domain.

This representation is efficient because tuples are immutable and can be used as dictionary keys. As a consequence, program execution becomes a sequence of dictionary lookups.

6.4 Parsing algorithm

The parser works in the following order:

1. normalize the text and split it into the `[domain]` and `[queries]` sections,
2. parse every domain line as one of the supported statement forms: `initially ...`, `... causes ... if ...`, `... costs ...`, or `... after ...`,
3. parse every query line as a goal query, a bounded-cost query, or an exact-cost query,
4. infer the complete signature of fluents and actions from the parsed domain.

The language is small and regular, so direct pattern-based parsing is simpler and more transparent than using a general parser generator.

6.5 Model construction algorithm

Let n be the number of fluents. The compiler first generates all 2^n possible complete Boolean assignments. Each assignment is treated as a candidate initial state.

For every action and every state, the next state is computed as follows:

1. start from the current state and apply the inertia law,
2. collect all effect statements relevant to the given action and fluent,
3. keep only effects whose preconditions are satisfied,
4. if no effect is applicable, preserve the old value,
5. if exactly one truth value is enforced, update the fluent,
6. if both positive and negative effects apply to the same fluent, report a contradiction.

After the transition table is built, the cost table is constructed. If an action has a declared cost and its execution changes the state, that cost is used; otherwise the cost is 0. This directly implements the empty-effect rule from the formal semantics.

6.6 Model filtering and query evaluation

Not every candidate initial state is a valid model. The compiler removes a state if:

- it violates an `initially` statement, or
- after executing a program required by a value statement, the resulting state does not satisfy the required literal.

The remaining states form the model set of the domain. Queries are then evaluated universally over this set:

- a goal query is true only if the goal holds in the final state of every model,
- a bounded-cost query is true only if the total cost never exceeds the bound,
- an exact-cost query is true only if every model produces exactly the required cost.

If no model remains after filtering, the compiler reports an inconsistent domain and does not print query results.

6.7 Complexity

The dominant cost of the algorithm is state-space generation. Since the number of candidate states is 2^n, the implementation is exponential in the number of fluents. For this reason it is intended mainly for small and medium-sized educational examples. The advantage of this exhaustive method is that it stays very close to the formal semantics and is easy to verify.

7 Testing

Michł Szewczak

Example 1: The Vault Room

Let's consider a scenario involving a vault room protected by wards and physical security layers and a thief that wants to access the vault illegally. He was given a key, but is not sure whether it is the key to the vault he plans to rob, and has no way to check it. Smashing the door is loud and it triggers the alarm and costs physical effort, but it always opens the door. Picking the lock is elegant and quiet, but it requires the key the thief has to be the right one for the lock to work and bypass the vault's wards, otherwise it completely fails.

Action Domain Statements

$$\text{initially } \neg doorUnlocked$$

$$\text{initially } \neg alarmActive$$

$$doorUnlocked \text{ after } pickLock, smashDoor$$

$$pickLock \text{ causes } doorUnlocked \text{ if } hasKey$$

$$smashDoor \text{ causes } doorUnlocked$$

$$smashDoor \text{ causes } alarmActive$$

$$pickLock \text{ costs } 2$$

$$smashDoor \text{ costs } 8$$

Program

$$P = (pickLock, smashDoor)$$

Completions of σ_0

Because the status of the fluent $hasKey$ is left unspecified by the initial historical assumptions, we must generate all possible complete initial states:

$$\sigma_0^1 = \{\neg doorUnlocked, \neg alarmActive, hasKey\}$$

$$\sigma_0^2 = \{\neg doorUnlocked, \neg alarmActive, \neg hasKey\}$$

However, σ_0^2 is not consistent with the value statement constraint in the action domain. Let us evaluate its branch:

1. **Action 1 (*pickLock*):** Since $hasKey$ does not hold in σ_0^2, the conditional effect does not apply. By the law of inertia, the state remains completely unchanged:

$$\Psi(pickLock, \sigma_0^2) \models \neg doorUnlocked$$

2. **Action 2 (*smashDoor*):** The execution continues to the final state:

$$\Psi^\star((pickLock, smashDoor), \sigma_0^2) \models doorUnlocked$$

But our structural rules dictate that a multi-step value statement f after $A_1, \ldots, A_n$ enforces satisfaction at *every* intermediary path step where that action prefix sequence resolves. The constraint $doorUnlocked$ after $pickLock, smashDoor$ demands that:

$$\Psi(pickLock, \sigma_0^2) \models doorUnlocked$$

This yields a direct contradiction with our step 1 transition evaluation. Therefore, the completion σ_0^2 is invalid and is completely filtered out of the model construction phase.

Execution

Thus, the only valid, consistent initial state structure is:

$$\sigma_0 = \{\neg doorUnlocked, \neg alarmActive, hasKey\}$$

Action 1: *pickLock* The precondition $hasKey$ holds true, meaning the postcondition triggers successfully:

$$\sigma_1 = \{doorUnlocked, \neg alarmActive, hasKey\}$$

Cost calculation: Since $\Psi(pickLock, \sigma_0) \neq \sigma_0$, the full cost statement maps:

$$Cost(pickLock, \sigma_0) = 2$$

Action 2: *smashDoor* Both effects of the smashing operation execute unconditionally:

$$\sigma_2 = \{doorUnlocked, alarmActive, hasKey\}$$

Cost calculation: While $doorUnlocked$ was already true and remains unchanged, the fluent $alarmActive$ transitions from false to true. Because a state transition occurred ($\Psi(smashDoor, \sigma_1) \neq \sigma_1$), the empty effect rule does not apply:

$$Cost(smashDoor, \sigma_1) = 8$$

Total Program Cost

$$Cost^\star(P) = 2 + 8 = 10$$

Final States

$$\{\sigma_2\}$$

Query Results

Q1: Goal Satisfaction

$$\{doorUnlocked, alarmActive\} \text{ after } pickLock, smashDoor$$

is true because both literals explicitly hold in the singular final state space model σ_2.

Q2: Executability with Cost

$$pickLock, smashDoor \text{ executable with cost } 10$$

is true since for all valid executions:

$$Cost^{\star}(P) \leq 10$$

Q3: Executability with exact Cost

$$pickLock, smashDoor \text{ executable with exact cost } 10$$

is true because the alternative inconsistent branch was eliminated, ensuring that $Cost^{\star}(P) = 10$ holds deterministically across all true models of the domain.

Print screen

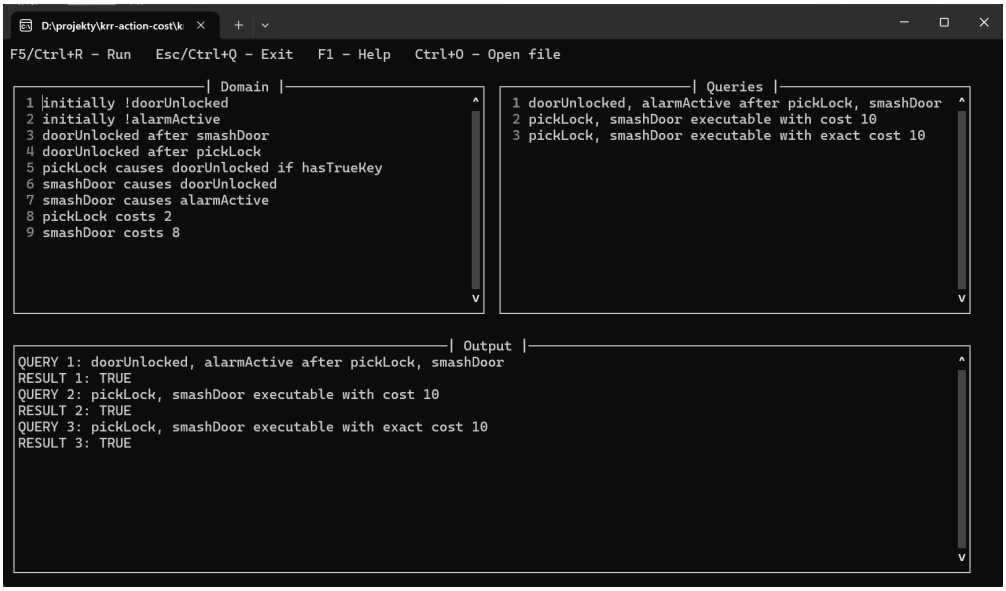

Figure 1: Michał Szewczak - example 1

7.1 Example 2: The Smart Greenhouse

Let us consider a scenario in an automated greenhouse where actions may yield empty effects depending on environmental resources.

7.1.1 Action Domain Statements

$$\text{initially } soilDry$$

$$\text{initially } \neg pumpActive$$

$$startPump \text{ causes } \neg soilDry \text{ if } \neg waterLow$$

$$startPump \text{ causes } pumpActive \text{ if } \neg waterLow$$

$$checkReservoir \text{ causes } \neg waterLow$$

$$startPump \text{ costs } 5$$

$$checkReservoir \text{ costs } 2$$

7.1.2 Program

$$P = (startPump, checkReservoir)$$

7.1.3 Completions of σ_0

Since the status of $waterLow$ is omitted from the initial historical properties, the system branches into two valid complete states:

$$\sigma_0^1 = \{soilDry, \neg pumpActive, \neg waterLow\}$$

$$\sigma_0^2 = \{soilDry, \neg pumpActive, waterLow\}$$

7.1.4 Execution Path 1 (σ_0^1)

Action 1 ($startPump$): The precondition $\neg waterLow$ holds true. The effects trigger successfully:

$$\sigma_1^1 = \{\neg soilDry, pumpActive, \neg waterLow\}$$

Cost calculation: A state change occurred ($\Psi(startPump, \sigma_0^1) \neq \sigma_0^1$), so the full cost is applied:

$$Cost(startPump, \sigma_0^1) = 5$$

Action 2 ($checkReservoir$): The action attempts to ensure water is not low. Because $\neg waterLow$ is already true, the state remains completely identical:

$$\sigma_2^1 = \{\neg soilDry, pumpActive, \neg waterLow\}$$

Cost calculation: Since $\Psi(checkReservoir, \sigma_1^1) = \sigma_1^1$, it results in an empty effect, driving the cost to zero:

$$Cost(checkReservoir, \sigma_1^1) = 0$$

$$\text{Total } Cost^\star(P) = 5 + 0 = 5$$

7.1.5 Execution Path 2 (σ_0^2)

Action 1 (*startPump*): The precondition $\neg waterLow$ fails. By the law of inertia, no fluents change:

$$\sigma_1^2 = \{soilDry, \neg pumpActive, waterLow\}$$

Cost calculation: Because $\Psi(startPump, \sigma_0^2) = \sigma_0^2$, the empty effect rule dictates the cost drops to zero:

$$Cost(startPump, \sigma_0^2) = 0$$

Action 2 (*checkReservoir*): The effect triggers unconditionally, shifting $waterLow$ to false:

$$\sigma_2^2 = \{soilDry, \neg pumpActive, \neg waterLow\}$$

Cost calculation: The state successfully changed, forcing the cost assignment:

$$Cost(checkReservoir, \sigma_1^2) = 2$$

$$\text{Total } Cost^\star(P) = 0 + 2 = 2$$

7.1.6 Query Results

$$P \text{ executable with cost } 4 \quad \text{(False, because } 5 \leq 4 \text{ and } 2 \leq 4\text{)}$$

$$P \text{ executable with exact cost } 4 \quad \text{(False, because program is not even executable with cost 4)}$$

Print screen

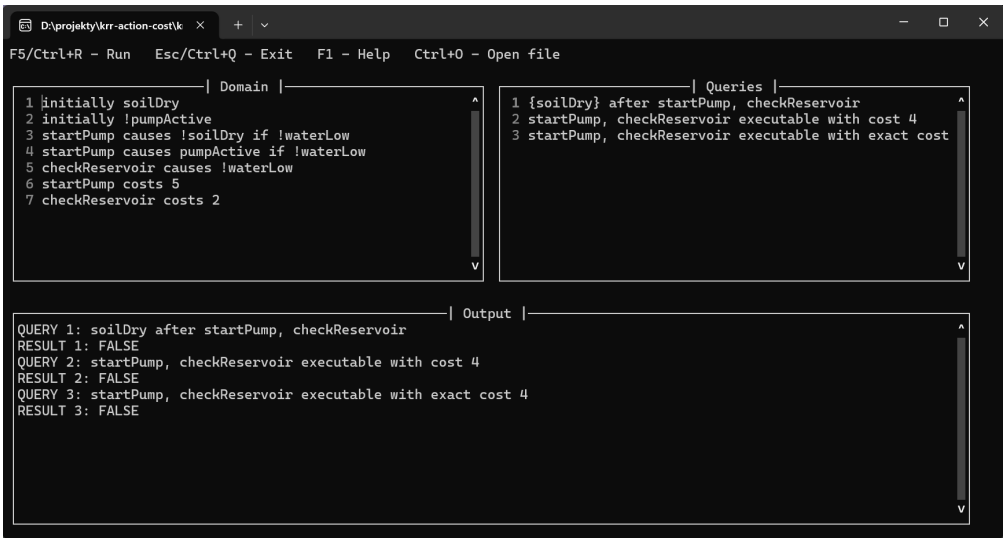

Figure 2: Michał Szewczak - example 2

7.2 Example 3: The Jammed Door via Single-Action Constraints (Inconsistent Domain)

This example refines the jammed mechanism scenario by using strictly single-action value statements to enforce a historical observation midway through the plan execution timeline. This formulation exposes the physical impossibility of the system state trajectory, allowing the solver to correctly flag the domain as inconsistent.

7.2.1 Action Domain Statements

$$\text{initially } \neg doorOpen$$

$$\text{initially } \neg doorUnlocked$$

$$\text{initially } mechanismJammed$$

$$doorUnlocked \text{ after } unlockDoor$$

$$unlockDoor \text{ causes } doorUnlocked \text{ if } \neg mechanismJammed$$

$$forceOpen \text{ causes } doorOpen \text{ if } doorUnlocked$$

$$unlockDoor \text{ costs } 3$$

$$forceOpen \text{ costs } 5$$

7.2.2 Program

$$P = (unlockDoor, forceOpen)$$

7.2.3 Completions of σ_0

The domain completely specifies the values of all system fluents at the origin of time. Hence, the initial configuration generates a single complete state representation:

$$\sigma_0 = \{\neg doorOpen, \neg doorUnlocked, mechanismJammed\}$$

7.2.4 Execution and Consistency Checking

We evaluate the state transition step-by-step from σ_0 under the action rules:

1. **Action 1 ($unlockDoor$):** The system attempts to process the transition $\Psi(unlockDoor, \sigma_0)$. The associated effect rule requires that $\neg mechanismJammed$ holds true. However, because $mechanismJammed$ is true in σ_0, the precondition fails. By the law of inertia, the action produces an empty effect, yielding:

$$\sigma_1 = \Psi(unlockDoor, \sigma_0) = \{\neg doorOpen, \neg doorUnlocked, mechanismJammed\}$$

Next, the semantic framework validates the structure against the action language value statement constraints. The domain includes the single-action observation statement:

$$doorUnlocked \text{ after } unlockDoor$$

By semantic definition, a structure satisfies this statement if and only if the fluent holds in the state immediately following the action's execution:

$$\Psi(unlockDoor, \sigma_0) \models doorUnlocked$$

However, the transition rules evaluated via inertia explicitly proved that $\sigma_1 \models \neg doorUnlocked$. Because a valid state assignment cannot simultaneously contain a fluent and its logical negation, no transition function can reconcile the observation constraint with the environmental physics.

Therefore, no valid structure exists for this setup, and the system correctly terminates during compilation with a status of:

DOMAIN STATUS: inconsistent

7.2.5 Query Results

Because no model can be constructed to satisfy the underlying constraints of the dynamic domain, all downstream query validations are structurally precluded from running. The query state evaluations are formally defined as **undefined**.

Print screen

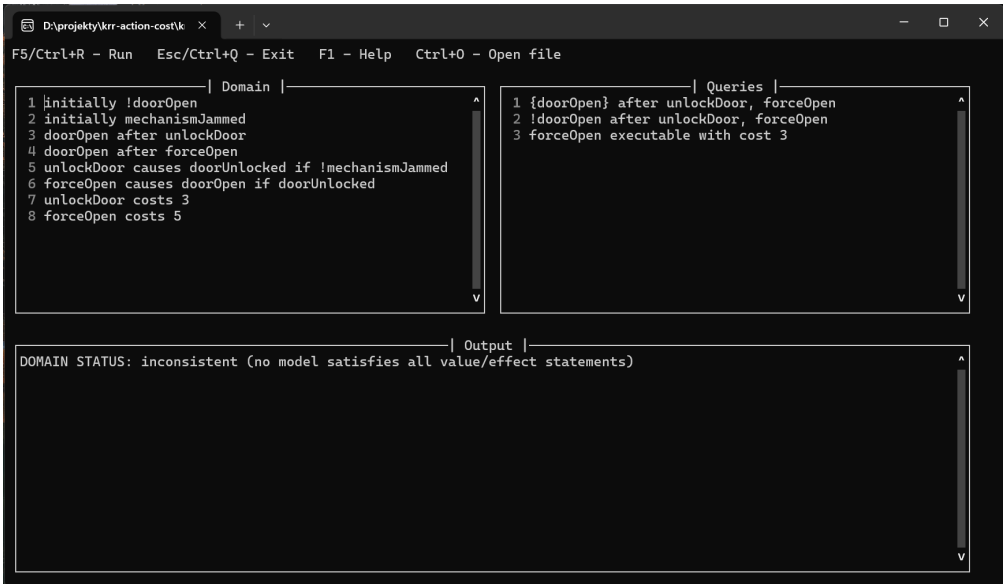

Figure 3: Michał Szewczak - example 3

Jakub Seliga

Example 1: The Light Switch

Let's consider a scenario with a light controlled by a switch that can be flipped off and back on, and which can also be permanently locked. Flipping the switch only works while the switch is unlocked; once locked, any further flip has no effect at all.

Action Domain Statements

initially $lightOn$

initially $\neg switchLocked$

$flipOff$ causes $\neg lightOn$ if $\neg switchLocked$

$flipOn$ causes $lightOn$ if $\neg switchLocked$

$lockSwitch$ causes $switchLocked$

$flipOff$ costs 3

$flipOn$ costs 4

$lockSwitch$ costs 1

Completions of σ_0

Both fluents are fixed by the initial statements, so the initial state is complete and the model is unique:

$$\sigma_0 = \{lightOn, \neg switchLocked\}$$

Execution

Program $(flipOff, flipOn)$ The precondition $\neg switchLocked$ holds, so $flipOff$ turns the light off:

$$\sigma_1 = \{\neg lightOn, \neg switchLocked\}$$

$$Cost(flipOff, \sigma_0) = 3$$

The precondition still holds, so $flipOn$ turns the light back on:

$$\sigma_2 = \{lightOn, \neg switchLocked\}$$

$$Cost(flipOn, \sigma_1) = 4$$

$$Cost^\star(P) = 3 + 4 = 7$$

Program $(flipOff, flipOff)$ The first $flipOff$ costs 3 as above. The second $flipOff$ finds $lightOn$ already false, so no change occurs:

$$Cost(flipOff, \sigma_1) = 0$$

$$Cost^\star(P) = 3 + 0 = 3$$

Program $(lockSwitch, flipOff)$ The action $lockSwitch$ sets the fluent $switchLocked$:

$$\sigma_1 = \{lightOn, switchLocked\}$$

$$Cost(lockSwitch, \sigma_0) = 1$$

Now the precondition $\neg switchLocked$ of $flipOff$ is violated, so the action has an empty effect and the empty effect rule applies:

$$\sigma_2 = \{lightOn, switchLocked\}$$

$$Cost(flipOff, \sigma_1) = 0$$

$$Cost^\star(P) = 1 + 0 = 1$$

Query Results

Q1: Goal Satisfaction

$$\{\neg lightOn\} \text{ after } flipOff$$

is true.

$$\{lightOn\} \text{ after } flipOff, flipOn$$

is true, since the light is turned off and then back on.

$$\{\neg lightOn\} \text{ after } lockSwitch, flipOff$$

is false, because locking the switch first turns the subsequent $flipOff$ into an empty effect and the light stays on.

Q2 and Q3: Executability with Cost

$$flipOff, flipOn \text{ executable with exact cost } 7$$

is true since $Cost^{\star}(P) = 7$.

$$flipOff, flipOff \text{ executable with exact cost } 3$$

is true, because the repeated $flipOff$ has an empty effect and adds zero cost.

$$lockSwitch, flipOff \text{ executable with exact cost } 1$$

is true, because the blocked $flipOff$ contributes no cost.

Print screen

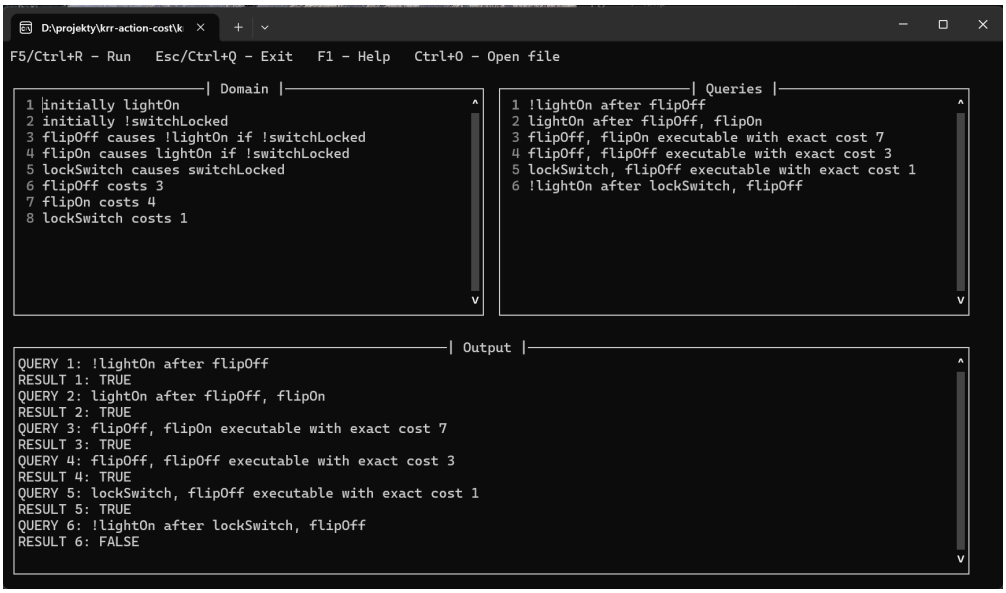

Figure 4: Jakub Seliga - example 1

Example 2: The Heater

Let's consider a heater that warms a room, but only when the room is powered. The power status is unknown in the initial state, and there is no value statement to settle it, so both possibilities must be considered.

Action Domain Statements

$$\text{initially } \neg heated$$

$$warmUp \text{ causes } heated \text{ if } powered$$

$$warmUp \text{ costs } 6$$

Program

$$P = (warmUp)$$

Completions of σ_0

The fluent *powered* is left unspecified, so two completions arise, and both are consistent since no value statement filters either out:

$$\sigma_0^1 = \{\neg heated, powered\}$$

$$\sigma_0^2 = \{\neg heated, \neg powered\}$$

Execution

Case 1: σ_0^1 Precondition holds, so:

$$\sigma_1^1 = \{heated, powered\}$$

Cost:

$$Cost^\star(P) = 6$$

Case 2: σ_0^2 Precondition does not hold, so:

$$\sigma_1^2 = \{\neg heated, \neg powered\}$$

Cost:

$$Cost^\star(P) = 0$$

Final States

$$\{\sigma_1^1, \sigma_1^2\}$$

Query Results

Q1: Goal Satisfaction

$$\{heated\} \text{ after } warmUp$$

is false because *heated* does not hold in all final states.

$$\{heated\} \text{ after } warmUp, warmUp$$

is false for the same reason.

Q2 and Q3: Executability with Cost

$$P \text{ executable with cost } 6$$

is true since for all executions:

$$Cost^\star(P) \leq 6$$

$$P \text{ executable with exact cost } 6$$

is false since $Cost^\star(P)$ is not always 6.

$$warmUp, warmUp \text{ executable with cost } 6$$

is true, because the second $warmUp$ is idempotent in the powered branch, so the total cost remains 6.

Print screen

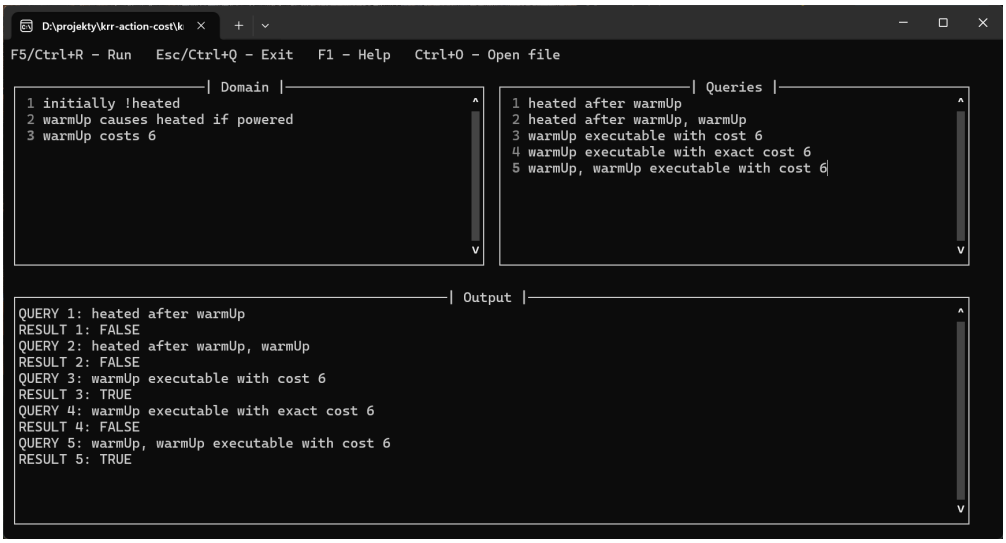

Figure 5: Jakub Seliga - example 2

Example 3: The Polishing Bench

Let's consider a workpiece that can be polished or buffed. Polishing requires the piece to be clean, and a value statement enforces that polishing always yields a polished piece. The cleanliness of the piece is unspecified initially, but the value statements collapse this uncertainty to a single model.

Action Domain Statements

$$\text{initially } \neg \mathit{polished}$$

$$\mathit{polished} \text{ after } \mathit{polish}$$

$$\mathit{polish} \text{ causes } \mathit{polished} \text{ if } \mathit{clean}$$

$$\mathit{clean} \text{ after } \mathit{polish}$$

$$\mathit{polish} \text{ costs } 4$$

$$\mathit{buff} \text{ causes } \mathit{polished}$$

$$\mathit{buff} \text{ costs } 2$$

Completions of σ_0

The fluent *clean* is unspecified, giving two candidate completions:

$$\sigma_0^1 = \{\neg \mathit{polished}, \mathit{clean}\}$$

$$\sigma_0^2 = \{\neg \mathit{polished}, \neg \mathit{clean}\}$$

However, σ_0^2 is not consistent with the action domain. Indeed, since *clean* does not hold, the conditional effect of *polish* does not apply, and by inertia:

$$\Psi(\mathit{polish}, \sigma_0^2) \models \neg \mathit{polished}$$

But the value statement requires:

$$\Psi(polish, \sigma_0^2) \models polished$$

This is a contradiction. Therefore, σ_0^2 is not allowed.

Execution

Thus, the only possible initial state is:

$$\sigma_0 = \{\neg polished, clean\}$$

Action *polish* The precondition *clean* holds, so *polished* becomes true while *clean* is untouched:

$$\sigma_1 = \{polished, clean\}$$

$$Cost(polish, \sigma_0) = 4$$

Repeated actions A second *polish*, or a following $buff$, finds *polished* already true, so no change occurs and the empty effect rule gives cost 0.

Final States

$$\{\sigma_1\}$$

Query Results

Q1: Goal Satisfaction

$$\{polished\} \text{ after } polish$$

is true.

$$\{clean\} \text{ after } polish$$

is true.

$$\{polished\} \text{ after } buff$$

is true.

Q2: Executability with Cost

$$polish \text{ executable with exact cost } 4$$

is true.

$$buff \text{ executable with exact cost } 2$$

is true.

Q3: Executability with exact Cost

$$polish, polish \text{ executable with exact cost } 4$$

is true, because the repeated *polish* re-asserts an already-true fluent and adds zero cost.

$$polish, buff \text{ executable with exact cost } 4$$

is true, since *buff* produces an empty effect after the piece is polished.

$$buff, buff \text{ executable with exact cost } 2$$

is true, for the same reason.

Print screen

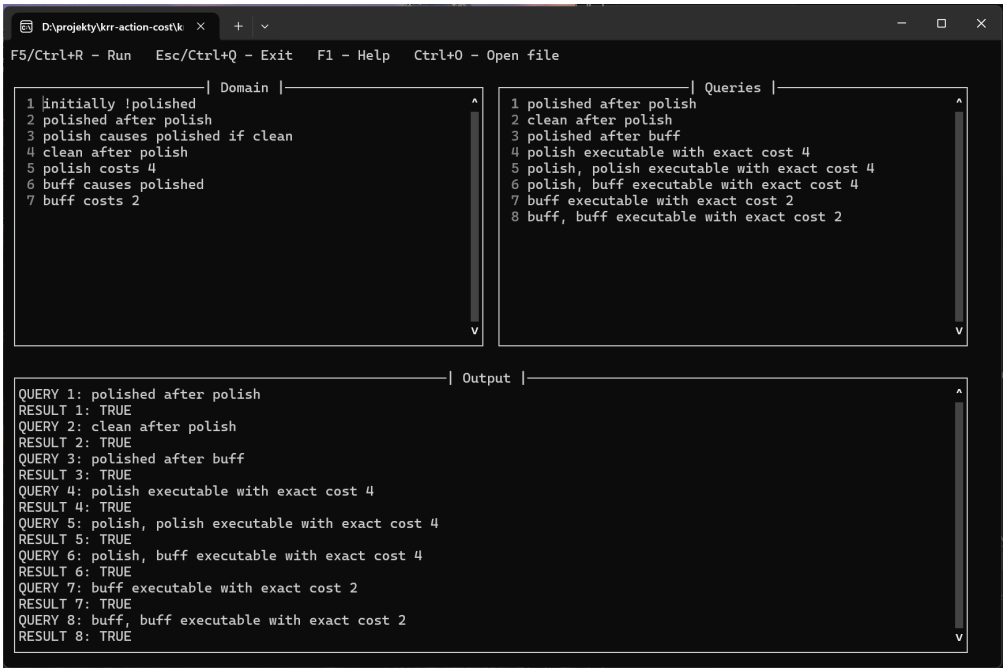

Figure 6: Jakub Seliga - example 3

8 User guide

The compiler is intended to be used through the prepared Windows executable. After launching the application, the user may either enter a specification directly in the editor or open an existing `.krr` or `.txt` file. The left pane is used for domain statements, the right pane for queries, and the output pane displays the compilation result.

8.1 Specification format

A stored specification should contain two sections:

```
[domain]
... domain statements ...

[queries]
... query statements ...
```

The domain section may contain:

- `initially fluent` or `initially !fluent`
- `fluent after action_1, action_2, ...`
- `action causes fluent if fluent, !fluent, ...`
- `action causes fluent`
- `action costs n`

The queries section may contain:

- goal queries of the form `fluent, !fluent after action_1, action_2, ...`
- bounded-cost queries of the form `action_1, action_2 executable with cost n`
- exact-cost queries of the form `action_1, action_2 executable with exact cost n`

Negation is written only with the symbol `!`. Comments begin with `#`. Files may use either the `.krr` or `.txt` extension.

8.2 Using the application

The normal workflow is:

- launch the executable,
- type the domain in the left pane and the queries in the right pane, or open an existing specification file with `Ctrl+O`,
- press `F5` or `Ctrl+R` to run the compiler,
- read the result in the output pane.

The most important shortcuts are:

- `F5` or `Ctrl+R` – run the compiler,
- `Esc` or `Ctrl+Q` – close the editor,
- `F1` – show or hide help,
- `Ctrl+O` – open an existing `.krr` or `.txt` file,
- `Tab` and `Shift+Tab` – switch between panes.

The domain may remain unchanged while the user edits the query pane and reruns the compiler multiple times. This makes the application convenient for testing many queries against the same action domain.

8.3 Output interpretation

For a consistent domain, the compiler prints every query together with the corresponding result `TRUE` or `FALSE`. If the domain is inconsistent, the compiler prints only:

```
DOMAIN STATUS: inconsistent
```

and query evaluation is skipped.

9 Contribution

For the parts evaluated in this report, the work was divided equally among all team members. The testing part was graded separately and is therefore not included in the division below. The following list presents the primary responsibility assigned to each person in the project.

- **Jakub Seliga – 25%**: preparation of the example part of the report, including the construction and description of example domains and illustrative scenarios.
- **Michal Szewczak – 25%**: preparation of the introduction, technical description of the adopted data structures and algorithms and, the user guide.
- **Sebastian Trojan – 25%**: implementation of the compiler and executable application.
- **Wiktoria Koniecko – 25%**: preparation and refinement of the formal theoretical part of the report, especially the syntax and semantics of the action language and query language, as well as notation consistency.